DevOps AWS Technical Assignment Report

1. Project Objective:

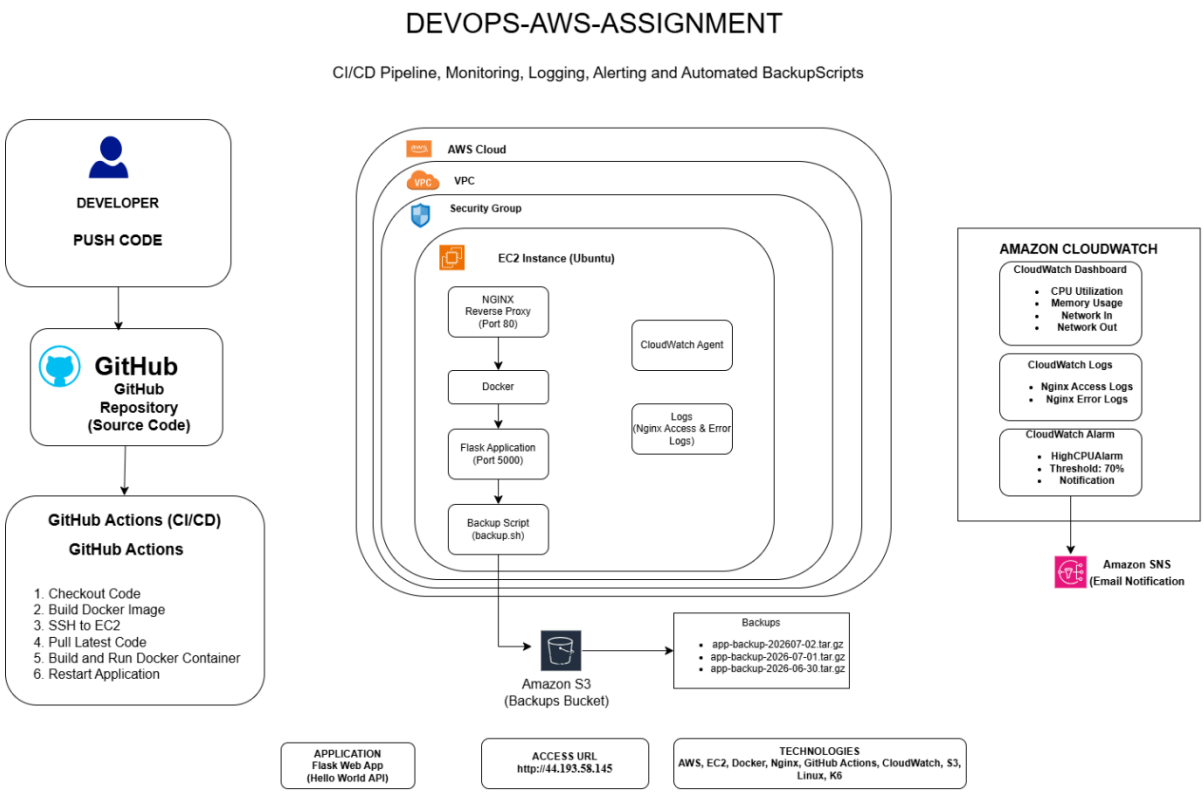
The objective of this project was to design, deploy, secure, monitor, and automate a production-like web application on AWS Free Tier. The project demonstrates practical DevOps skills by deploying a Dockerized Flask application on an EC2 instance, implementing CI/CD using GitHub Actions, configuring monitoring with Amazon CloudWatch, automating backups using Amazon S3, and evaluating application performance through load testing.

2. Project Overview:

This project implements a complete DevOps workflow on AWS. A Flask-based web application was containerized using Docker and deployed on an Ubuntu EC2 instance. Nginx was configured as a reverse proxy to expose the application. GitHub Actions automatically deploys new changes to the EC2 instance whenever code is pushed to the main branch.

CloudWatch Agent was configured to collect application and system metrics. Monitoring dashboards, alarms, and log collection were implemented to observe application health. Automated backups were uploaded to Amazon S3 using a shell script. Finally, k6 was used to perform load testing and analyze application performance.

3. Architecture



Architecture Diagram

Architecture Flow:

Developer

↓

GitHub Repository

↓

GitHub Actions (CI/CD)

↓

AWS EC2 Instance (Ubuntu)

↓

Nginx Reverse Proxy

↓

Docker Container

↓

Flask Application

↓

CloudWatch Monitoring & Logs

↓

Amazon S3 Backups

4. AWS Services Used:

The following AWS services were used in this project:

- Amazon EC2
- Amazon IAM
- Amazon S3
- Amazon CloudWatch

5. Technologies Used:

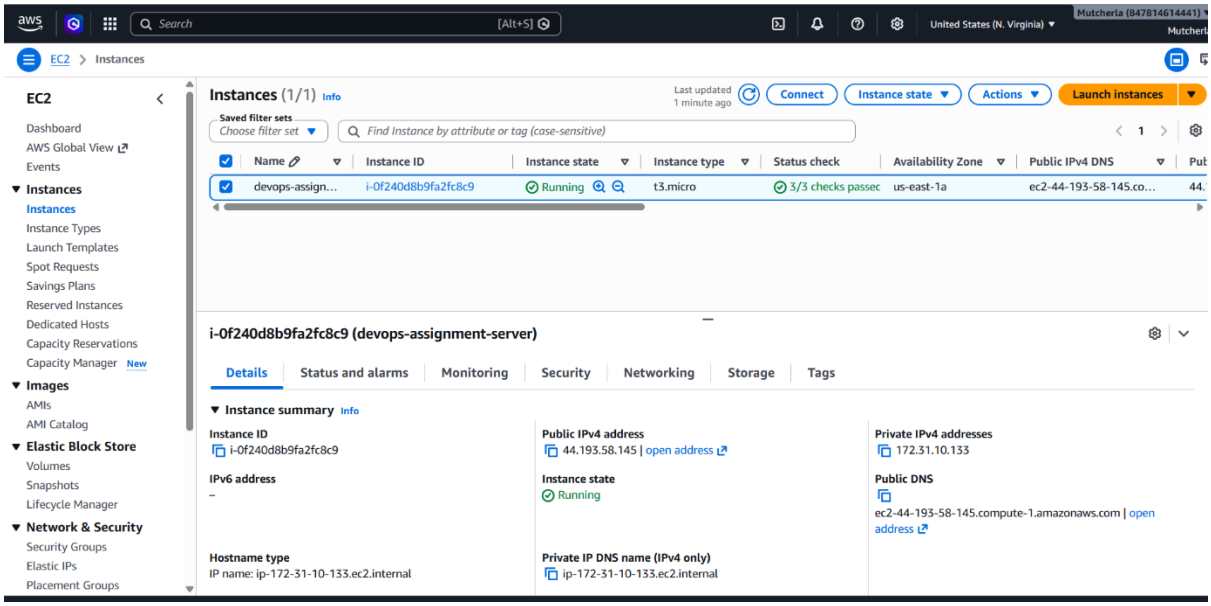
- Ubuntu Server
- Docker

- Flask
- Nginx
- Git
- GitHub Actions
- AWS CLI
- Amazon CloudWatch Agent
- k6
- Linux Shell Scripting

6. Infrastructure Setup:

An Ubuntu EC2 instance was launched using AWS Free Tier. Security Groups were configured to allow SSH (Port 22), HTTP (Port 80), and application traffic where required. An IAM role following the principle of least privilege was attached to the EC2 instance to allow secure access to Amazon S3 and CloudWatch.

Docker, Nginx, Git, AWS CLI, and the CloudWatch Agent were installed on the EC2 instance. The application source code was cloned from GitHub, and the Flask application was built as a Docker image and deployed inside a container.

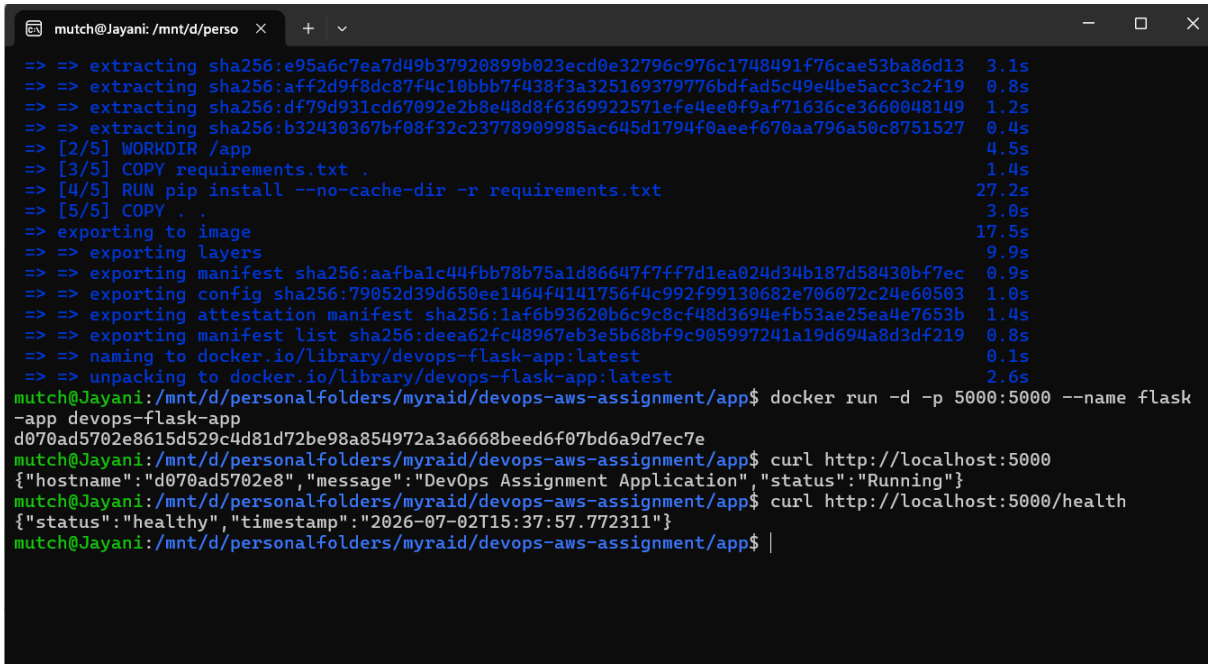


7. Application Deployment

The Flask application was containerized using Docker.

Deployment process:

- Clone the GitHub repository.
- Build the Docker image.
- Run the Docker container.
- Configure Nginx as a reverse proxy.
- Verify application availability through the EC2 public IP.



8. CI/CD Pipeline:

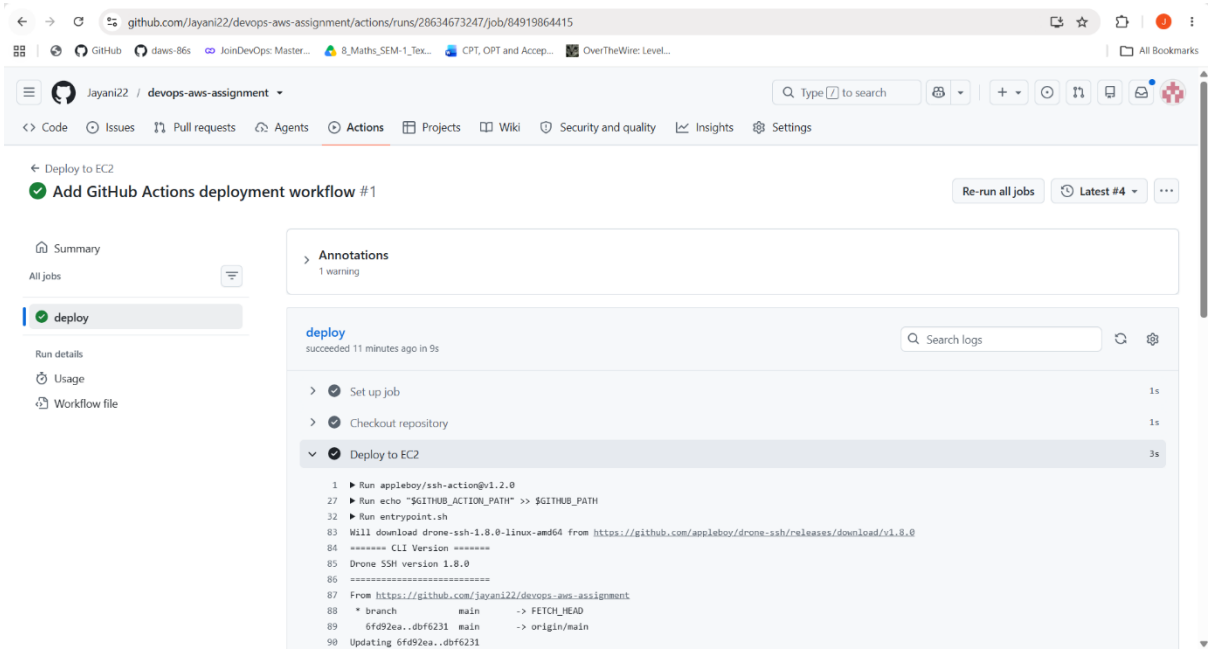
GitHub Actions was configured to automate deployments.

Pipeline workflow:

1. Developer pushes code to GitHub.
2. GitHub Actions workflow starts automatically.
3. Workflow connects to the EC2 instance using SSH.
4. Latest code is pulled from GitHub.
5. Docker image is rebuilt.

6. Existing container is stopped and removed.
7. A new container is started with the latest application.

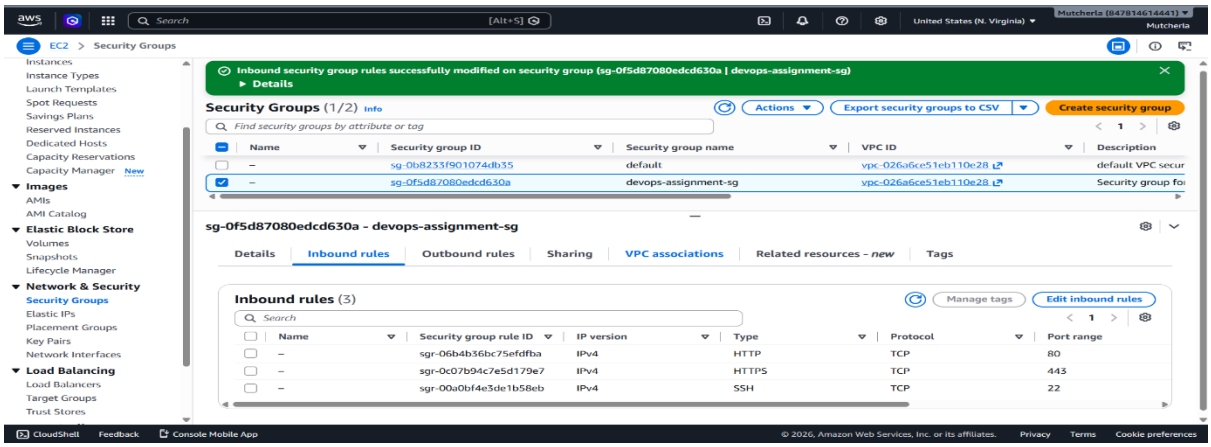
This eliminates manual deployment and ensures consistent releases.

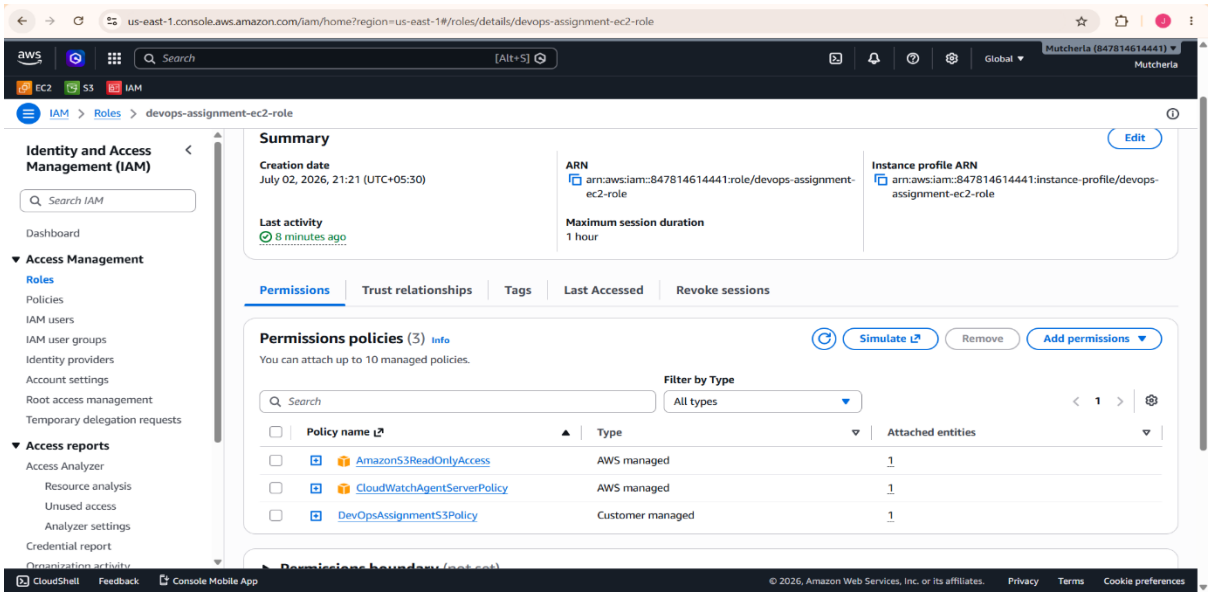


9. Security Configuration:

The following security best practices were implemented:

- IAM role with least privilege permissions.
- SSH key authentication for EC2 access.
- Security Groups configured to allow only required ports.
- GitHub Secrets used for storing deployment credentials.
- Private Amazon S3 bucket used for backups.
- Docker containers used for application isolation.





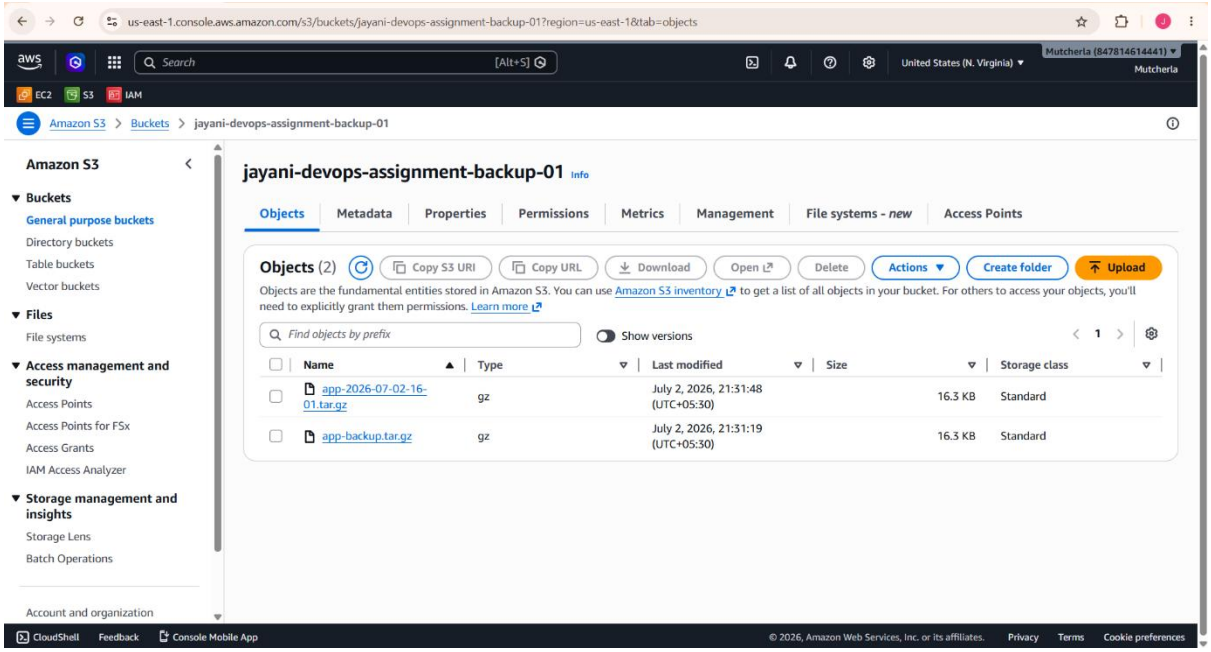
10. Backup Strategy:

A shell script was created to automate application backups.

The backup process performs the following tasks:

- Compresses the application into a tar.gz archive.
- Generates timestamped backup files.
- Uploads backups to Amazon S3 using the AWS CLI.

This ensures application data can be restored if required.



11. Monitoring:

Amazon CloudWatch Agent was configured to monitor the EC2 instance.

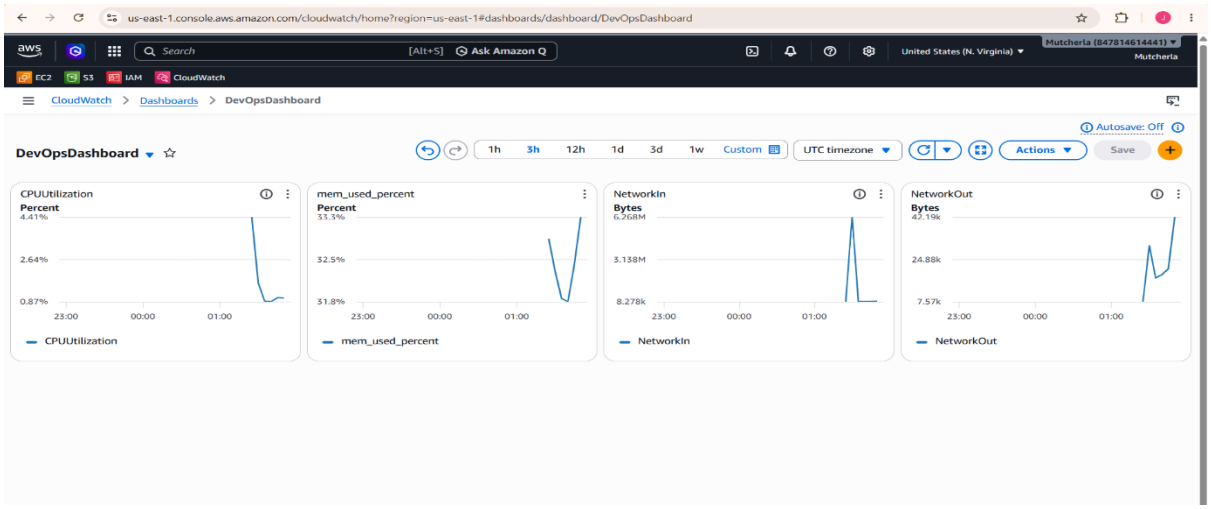
The dashboard includes:

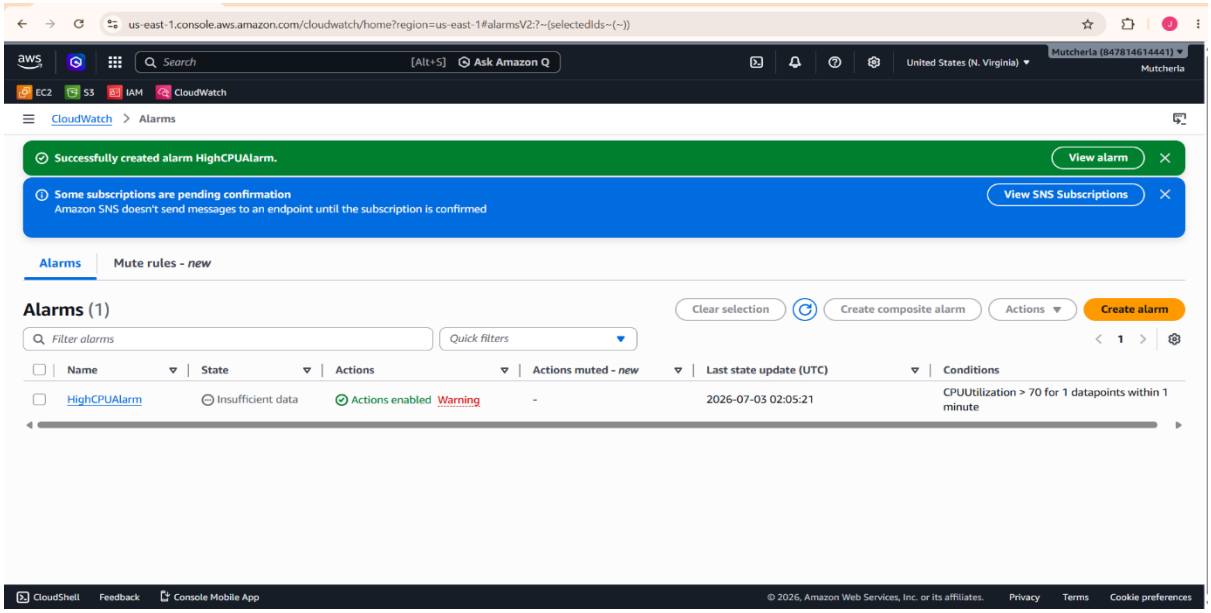
- CPU Utilization
- Memory Usage
- Network In
- Network Out

CloudWatch Logs were configured to collect:

- Nginx Access Logs
- Nginx Error Logs

A CloudWatch Alarm was created to notify when CPU utilization exceeds 70%.





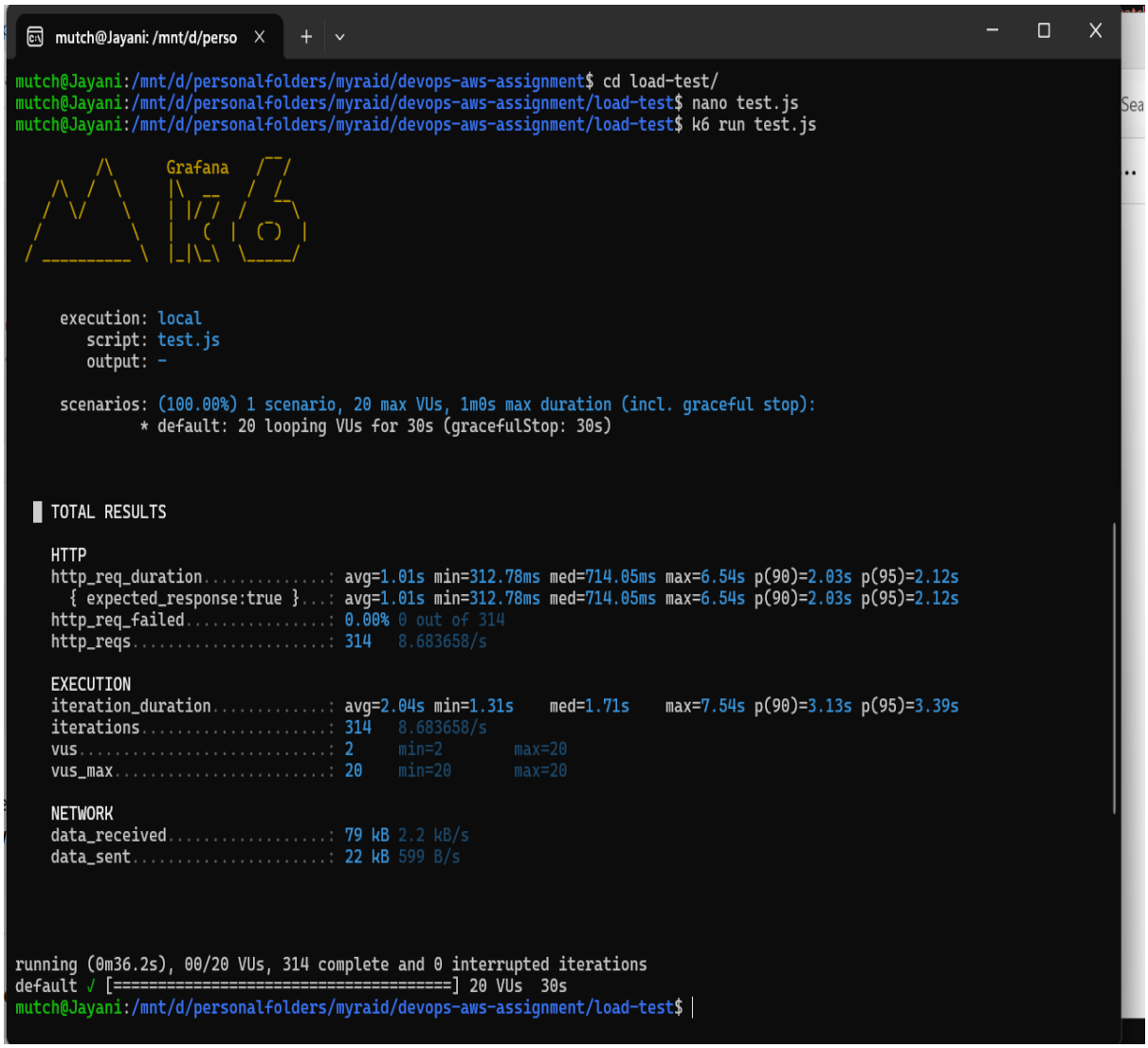
12. Load Testing:

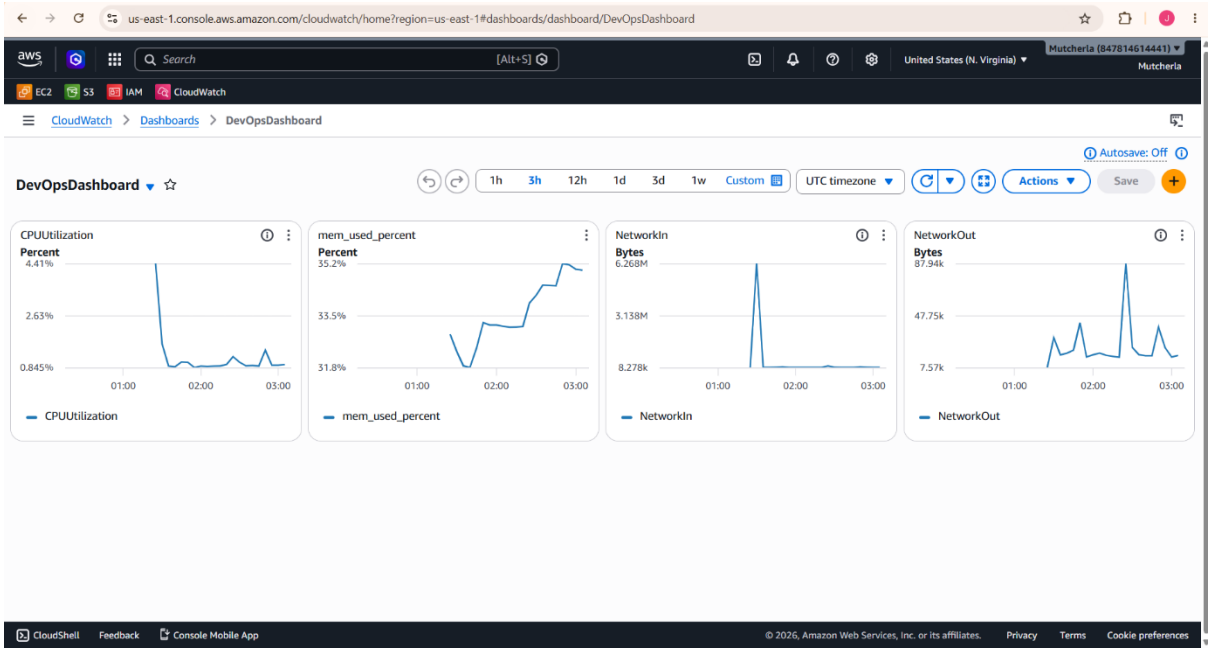
Load testing was performed using **k6**.

The following metrics were monitored:

- Response Time
- Throughput
- Requests per Second
- Error Rate
- CPU Utilization
- Memory Usage
- Network Activity

During testing, the application remained responsive and handled concurrent requests successfully. CloudWatch metrics showed increased CPU and network usage during the test while maintaining stable application performance.





13. Performance Analysis:

Observations

- Application responded successfully under load.
- CPU utilization increased during testing but remained within acceptable limits.
- Memory usage remained stable.
- Network traffic increased as expected.
- No significant application errors were observed.

Suggested Improvements

- Enable Auto Scaling.
- Configure an Application Load Balancer.
- Implement HTTPS using AWS Certificate Manager.
- Provision infrastructure using Terraform.
- Deploy the application on Kubernetes for high availability.

14. Challenges Faced:

During the implementation of this project, several challenges were encountered:

- SSH private key permission issues while connecting to the EC2 instance from WSL.
- IAM permission errors (s3:PutObject) while uploading backups to Amazon S3.
- CloudWatch Agent configuration and verification of custom metrics.
- GitHub Actions deployment timeout caused by Security Group SSH restrictions.

Each issue was resolved through proper configuration and verification of AWS services.

15. Project Deliverables:

The project repository contains:

- Source Code
- Docker Configuration
- GitHub Actions Workflow
- CloudWatch Configuration
- Backup Script
- Nginx Configuration
- Load Testing Script
- Architecture Diagram
- Screenshots
- README Documentation

16. Conclusion:

This project successfully demonstrates an end-to-end DevOps workflow using AWS Free Tier services. A Dockerized Flask application was deployed on an EC2 instance and automated through GitHub Actions. Monitoring, logging, alarms, and backup mechanisms were implemented using Amazon CloudWatch and Amazon S3. Load testing confirmed that the application performs reliably under concurrent requests.

The project strengthened practical knowledge of cloud infrastructure, containerization, CI/CD automation, monitoring, security, and performance testing, while following common DevOps best practices.